

TRƯỜNG KỸ THUẬT VÀ CÔNG NGHỆ
KHOA CÔNG NGHỆ THÔNG TIN



THỰC TẬP ĐỒ ÁN CƠ SỞ NGÀNH
HỌC KỲ I, NĂM HỌC 2025-2026

**THIẾT KẾ VÀ CÀI ĐẶT CƠ SỞ DỮ LIỆU
NoSQL CHO HỆ THỐNG
NGƯỜI DÙNG MẠNG XÃ HỘI**

Giảng viên hướng dẫn:

ThS.GVC Phan Thị Phương Nam

Sinh viên thực hiện:

Họ tên: Chung Mỹ Linh

MSSV: 110123023

Lớp: DA23TTA

Vĩnh Long, tháng 12 năm 2025

TRƯỜNG KỸ THUẬT VÀ CÔNG NGHỆ
KHOA CÔNG NGHỆ THÔNG TIN



THỰC TẬP ĐỒ ÁN CƠ SỞ NGÀNH
HỌC KỲ I, NĂM HỌC 2025-2026

**THIẾT KẾ VÀ CÀI ĐẶT CƠ SỞ DỮ LIỆU
NoSQL CHO HỆ THỐNG
NGƯỜI DÙNG MẠNG XÃ HỘI**

Giảng viên hướng dẫn:

ThS.GVC Phan Thị Phương Nam

Sinh viên thực hiện:

Họ tên: Chung Mỹ Linh

MSSV: 110123023

Lớp: DA23TTA

Vĩnh Long, tháng 12 năm 2025

NHẬN XÉT CỦA GIẢNG VIÊN HƯỚNG DẪN

[illegible]

Vĩnh Long, ngày tháng 01 năm 2026
Giảng viên hướng dẫn

Phan Thị Phương Nam

NHẬN XÉT CỦA THÀNH VIÊN HỘI ĐỒNG

[illegible]

Vĩnh Long, ngày tháng năm

Thành viên hội đồng
(Ký tên và ghi rõ họ tên)

LỜI CẢM ƠN

Em xin gửi lời cảm ơn chân thành và sâu sắc nhất đến cô Phan Thị Phương Nam, giảng viên hướng dẫn đã tận tình hỗ trợ, định hướng và góp ý trong suốt quá trình em thực hiện đề án với đề tài “Thiết kế và cài đặt cơ sở dữ liệu NoSQL cho hệ thống người dùng mạng xã hội”. Nhờ sự đồng hành, chỉ dẫn và kinh nghiệm chuyên môn sâu sắc của cô, em đã có thể tiếp cận, nghiên cứu chuyên sâu về kiến trúc NoSQL và hệ thống quản lý dữ liệu lớn một cách bài bản, khoa học và sáng tạo hơn.

Đồng thời, em xin cảm ơn quý thầy cô trong Khoa Công Nghệ Thông Tin đã truyền đạt những kiến thức nền tảng về thiết kế cơ sở dữ liệu, và các kỹ năng chuyên môn liên quan. Những kiến thức đó là nền tảng vô cùng quan trọng giúp em hoàn thành bài đề án này.

Cuối cùng, em xin cảm ơn các bạn học cùng lớp đã luôn hỗ trợ, chia sẻ tài liệu và trao đổi kinh nghiệm chuyên môn trong quá trình học tập và thực hiện đề tài.

Dù đã cố gắng hoàn thành với tinh thần nghiêm túc và trách nhiệm, nhưng do kiến thức và kinh nghiệm thực tế còn hạn chế, bài làm chắc chắn vẫn còn tồn tại những thiếu sót nhất định. Em kính mong nhận được sự thông cảm và những ý kiến đóng góp quý báu, chân thành từ quý thầy/cô để em có thể rút kinh nghiệm và hoàn thiện bản thân trong các nghiên cứu và đồ án chuyên môn sắp tới.

Trân trọng cảm ơn!

MỤC LỤC

TÓM TẮT ĐỒ ÁN CƠ SỞ NGÀNH	12
MỞ ĐẦU	13
CHƯƠNG 1: TỔNG QUAN	15
CHƯƠNG 2: NGHIÊN CỨU LÝ THUYẾT	16
2.1. Giới thiệu về NoSQL	16
2.1.1. Khái niệm	16
2.1.2. Các kiểu mô hình dữ liệu trong NoSQL	16
2.1.2.1. Key-Value Store	16
2.1.2.2. Document Store	16
2.1.2.3. Column-family Store	17
2.1.2.4. Graph Database	17
2.1.3. Các khái niệm trong cơ sở dữ liệu NoSQL	18
2.1.3.1. Schema-less / Flexible Schema (Lược đồ linh hoạt)	18
2.1.3.2. Document Database	18
2.1.3.3. Collection	18
2.1.3.4. Key-Value Store	18
2.1.3.5. Replication	18
2.1.3.6. Định lý CAP	18
2.1.3.7. Mô hình BASE	18
2.1.4. Một số ưu điểm và nhược điểm của cơ sở dữ liệu NoSQL	19
2.1.4.1. Ưu điểm:	19
2.1.4.2. Nhược điểm:	19
2.2. Giới thiệu về MongoDB	19
2.2.1. Tổng quan về MongoDB	19
2.2.2. Một số khái niệm cơ bản trong MongoDB	20
2.2.3. Các kiểu dữ liệu trong MongoDB	21
2.2.4. Các thao tác dữ liệu trên MongoDB	22

2.2.4.1. Tạo cơ sở dữ liệu	22
2.2.4.2. Kiểm tra cơ sở dữ liệu hiện hành	22
2.2.4.3. Kiểm tra danh sách các cơ sở dữ liệu	22
2.2.4.4. Xóa cơ sở dữ liệu	23
2.2.4.5. Tạo Collection	23
2.2.4.6. Xóa Collection	24
2.2.4.7. Thêm mới Document	24
2.2.4.8. Truy vấn Document	24
2.2.4.9. Cập nhật Document	24
2.2.4.10. Xóa Document trong MongoDB	25
2.2.4.11. Tạo backup	25
2.2.4.12. Phục hồi dữ liệu trong MongoDB	26
2.2.5. Cơ chế Index và Aggregation	26
CHƯƠNG 3: HIỆN THỰC HÓA NGHIÊN CỨU	28
3.1. Cài đặt và chuẩn bị môi trường	28
3.1.1. Hạ tầng phần mềm	28
3.1.2. Công cụ quản trị và thao tác	28
3.2. Lược đồ tổng quát cơ sở dữ liệu	28
3.3. Hiện thực hóa cấu trúc dữ liệu	30
3.3.1. Chiến lược thiết kế	30
3.3.2. Chi tiết cấu trúc Document của các Collection	31
3.3.2.1. Collection nguoi_dung (Collection Trung tâm)	31
3.3.2.2. Collection bai_dang	31
3.3.2.3. Collection binh_luan	32
3.3.2.4. Collection luot_thich	33
3.3.2.5. Collection tin_nhan	33
3.3.2.6. Collection thong_bao	34
3.3.2.7. Collection theo_doi	34
3.3.2.8. Collection media	35
3.3.2.9. Collection bao_cao	35

3.4 Khởi tạo và cấu hình cơ sở dữ liệu	35
CHƯƠNG 4: KẾT QUẢ NGHIÊN CỨU	37
4.1. Hiện thực hóa các nghiệp vụ	37
4.1.1. Nghiệp vụ cập nhật dữ liệu	37
4.1.1.1. Cập nhật thông tin người dùng	37
4.1.1.2. Cập nhật hàng loạt	38
4.1.2. Nghiệp vụ tra cứu và tìm kiếm	39
4.1.2.1. Tra cứu chi tiết	39
4.1.2.2. Tra cứu theo tiêu chí	40
4.1.2.3. Tra cứu nâng cao theo Văn bản	41
4.1.3. Nghiệp vụ thống kê và báo cáo	42
4.1.3.1. Thống kê 1 người dùng có bao nhiêu bài đăng	42
4.1.3.2. Thống kê 1 người có bao nhiêu bình luận	44
4.1.3.3. Thống kê bài đăng có bao nhiêu lượt thích và bình luận	46
4.2. Kết quả hiện thực hóa dữ liệu	47
4.3. Kết quả các giao diện chức năng trên MongoDB Compass	48
4.4. Kết quả các nghiệp vụ thống kê báo cáo	48
4.5. Đánh giá và Phân tích kết quả	48
4.5.1. Về hiệu năng và tốc độ truy xuất	48
4.5.2. Về tính linh hoạt và khả năng mở rộng	48
4.5.3. Trải nghiệm trong quá trình thực hiện	48
4.6. Ưu điểm và nhược điểm	49
4.6.1. Ưu điểm	49
4.6.2. Nhược điểm	49
CHƯƠNG 5: KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN	50
5.1. Kết luận	50
5.2. Hướng phát triển	50
DANH MỤC TÀI LIỆU THAM KHẢO	51

DANH MỤC HÌNH ẢNH

Hình 2.1 Minh họa Document trong Collection.....	20
Hình 3.1 Giao diện dòng lệnh khi khởi tạo Database và Collection.....	36
Hình 3.2 Giao diện MongoDB Compass hiển thị Database và các Collection đã tạo thành công.....	36

DANH MỤC BẢNG BIỂU

Bảng 2.1: Mối quan hệ giữa các thuật ngữ của RDBMS và MongoDB	21
Bảng 2.2: Tham số options của Collection	23
Bảng 2.3: Danh sách các tùy chọn options của Collection	24
Bảng 2.4: Các lệnh tùy chọn trong lệnh Backup	25
Bảng 3.1: Lược đồ cơ sở dữ liệu mức vật lý	28

DANH MỤC TỪ VIẾT TẮT

Từ viết tắt	Tên đầy đủ
ACID	Atomicity, Consistency, Isolation, Durability
API	Application Programming Interface
BASE	Basically Available, Soft state, Eventual consistency
BSON	Binary JSON
CAP	Consistency, Availability, Partition Tolerance
CMS	Content Management System
JSON	JavaScript Object Notation
JWT	JSON Web Token
MQL	MongoDB Query Language
NoSQL	Not only SQL
RDBMS	Relational Database Management System
SQL	Structured Query Language
UTF-8	Unicode Transformation Format - 8-bit
XML	eXtensible Markup Language

TÓM TẮT ĐỀ ÁN CƠ SỞ NGÀNH

Vấn đề nghiên cứu: Các hệ thống mạng xã hội hiện nay phải xử lý khối lượng dữ liệu người dùng ngày càng lớn và có tính đa dạng cao, bao gồm thông tin cá nhân, mối quan hệ bạn bè, bài đăng, bình luận, tương tác, tin nhắn và thông báo. Việc sử dụng mô hình cơ sở dữ liệu quan hệ truyền thống dần bộc lộ nhiều hạn chế như: cấu trúc lược đồ cứng nhắc, khó thích ứng khi yêu cầu nghiệp vụ thay đổi, khả năng mở rộng hạn chế và hiệu suất truy vấn suy giảm do phải thực hiện nhiều phép nối bảng phức tạp. Trước thực trạng đó, đề án tập trung nghiên cứu giải pháp thiết kế và cài đặt cơ sở dữ liệu NoSQL nhằm nâng cao hiệu quả quản trị, lưu trữ và khai thác dữ liệu cho hệ thống người dùng mạng xã hội.

Hướng tiếp cận: Đề tài lựa chọn hệ quản trị MongoDB – cơ sở dữ liệu dạng tài liệu (Document Store) phổ biến và phù hợp. Hướng tiếp cận chính là khai thác lược đồ linh hoạt và kỹ thuật nhúng dữ liệu, lấy người dùng làm trung tâm để tổ chức hiệu quả các thông tin liên quan và tối ưu các thao tác truy xuất.

Cách giải quyết vấn đề:

Thiết kế lược đồ: Xây dựng collection trung tâm `nguoi_dung`, tích hợp các thông tin như hồ sơ cá nhân, bài đăng, bình luận, bạn bè, lượt thích, tin nhắn, media, báo cáo vi phạm và thông báo bằng mô hình dữ liệu nhúng nhằm hạn chế truy vấn chéo.

Triển khai kỹ thuật: Hệ thống được triển khai và quản trị bằng MongoDB Shell và MongoDB Compass.

Hiện thực hóa nghiệp vụ: Xây dựng các truy vấn bằng ngôn ngữ MQL, với các thao tác cập nhật, cập nhật hàng loạt (`updateMany`) và tìm kiếm nâng cao bằng `Regex`.

Tối ưu hóa báo cáo: Aggregation Pipeline được áp dụng để thống kê số lượng người dùng, mức độ tương tác và tần suất đăng bài.

Kết quả đạt được:

Xây dựng thành công mô hình dữ liệu linh hoạt, để bổ sung các trường thông tin mới mà không làm gián đoạn hệ thống, cải thiện hiệu năng truy xuất nhờ giảm thiểu các phép Join phức tạp đáp ứng yêu cầu xử lý dữ liệu lớn của hệ thống người dùng mạng xã hội

MỞ ĐẦU

Đề tài “Thiết kế và cài đặt cơ sở dữ liệu NoSQL cho hệ thống người dùng mạng xã hội” tập trung vào việc xây dựng một mô hình cơ sở dữ liệu hiện đại, linh hoạt và dễ dàng mở rộng lược đồ nhằm phục vụ hiệu quả cho công tác quản lý, lưu trữ và khai thác dữ liệu người dùng trong các hệ thống mạng xã hội hiện nay.

Hệ thống mạng xã hội thường phải xử lý một khối lượng dữ liệu lớn và đa dạng, bao gồm thông tin hồ sơ người dùng, mối quan hệ bạn bè hoặc theo dõi, bài đăng, bình luận, lượt thích, tin nhắn, media và thông báo. Các dữ liệu này có đặc điểm thay đổi liên tục, không đồng nhất và khó dự đoán trước về cấu trúc, do đó việc tổ chức, lưu trữ và truy xuất dữ liệu đòi hỏi một mô hình cơ sở dữ liệu có tính linh hoạt cao, đồng thời đảm bảo hiệu năng và khả năng mở rộng khi hệ thống phát triển.

Thay vì tiếp tục dựa vào mô hình cơ sở dữ liệu quan hệ truyền thống với cấu trúc bảng ràng buộc chặt chẽ và yêu cầu thực hiện nhiều thao tác kết nối phức tạp để truy xuất dữ liệu liên quan, đề tài lựa chọn hướng tiếp cận mới bằng cách sử dụng cơ sở dữ liệu NoSQL. Sử dụng hệ quản trị cơ sở dữ liệu MongoDB để lưu trữ dữ liệu sẽ giúp cho việc quản lý dữ liệu trở nên trực quan hơn, đồng thời mang lại nhiều ưu điểm như:

- Khả năng mở rộng cấu trúc linh hoạt giúp hệ thống dễ dàng đáp ứng khi số lượng người dùng và dữ liệu tăng nhanh.
- Tốc độ truy xuất dữ liệu nhanh phù hợp với các hệ thống mạng xã hội có yêu cầu xử lý lớn và gần thời gian thực.
- Cấu trúc dữ liệu linh hoạt cho phép dễ dàng thay đổi hoặc bổ sung các thuộc tính mà không cần chỉnh sửa toàn bộ cấu trúc cơ sở dữ liệu.

Đề tài tập trung nghiên cứu mô hình dữ liệu của hệ thống người dùng mạng xã hội Facebook, từ đó tiến hành phân tích và thiết kế cơ sở dữ liệu theo hướng NoSQL. Sau đó, hệ thống được triển khai trên MongoDB, thực hiện nhập dữ liệu mẫu thử và xây dựng các truy vấn nhằm minh họa cách thức hoạt động của hệ thống quản lý người dùng mạng xã hội.

- Phương pháp thực hiện:

+ Nghiên cứu lý thuyết: Nghiên cứu các nghiệp vụ cơ bản của hệ thống mạng xã hội như quản lý hồ sơ người dùng, bài đăng, bình luận, tương tác, tin nhắn và mối quan hệ giữa người dùng. Đồng thời nghiên cứu các đặc trưng, ưu điểm và hạn chế của cơ sở dữ liệu NoSQL, tìm hiểu chi tiết về MongoDB bao gồm mô hình Document, Collection, các thao tác truy vấn, cơ chế Index và ngôn ngữ truy vấn MQL để thực hiện các truy vấn phức tạp.

+ Thiết kế cơ sở dữ liệu: Xác định các collection chính của hệ thống mạng xã hội; xác định thuộc tính của từng collection và mối liên kết giữa các collection theo mô hình dữ liệu NoSQL.

+ Cài đặt hệ thống và thử nghiệm: Cài đặt hệ quản trị cơ sở dữ liệu MongoDB trên máy tính cá nhân, tạo các collection theo thiết kế đã đề xuất, đảm bảo tính đúng đắn của cấu trúc, nhập dữ liệu mẫu thử để kiểm tra tính hiệu quả của mô hình, xây dựng và thực thi các truy vấn mẫu để minh họa các chức năng quản lý cốt lõi: thêm, xóa, sửa, tìm kiếm người dùng, tìm kiếm bài đăng, thống kê người dùng có bao nhiêu bài đăng,..., đối chiếu kết quả với yêu cầu nghiệp vụ của hệ thống mạng xã hội.

- Đối tượng nghiên cứu:

+ Hệ quản trị CSDL NoSQL: Nghiên cứu sâu về các đặc tính của CSDL Document Store và các công cụ kèm theo để triển khai và quản lý dữ liệu.

+ MongoDB: Hệ thống quản trị CSDL cụ thể được lựa chọn, bao gồm: cấu trúc dữ liệu BSON Schema, ngôn ngữ truy vấn MQL, cơ chế Index và Aggregation.

+ Nghiệp vụ và dữ liệu: Phân tích chi tiết các nghiệp vụ cốt lõi và các luồng dữ liệu liên quan đến quản lý người dùng và nội dung trong hệ thống mạng xã hội.

- Phạm vi nghiên cứu: Thiết kế lược đồ cơ sở dữ liệu NoSQL cho hệ thống người dùng mạng xã hội Facebook.

CHƯƠNG 1: TỔNG QUAN

Đề tài “Thiết kế và cài đặt cơ sở dữ liệu NoSQL cho hệ thống người dùng mạng xã hội” tập trung vào việc giải quyết các vấn đề trong công tác quản lý và lưu trữ dữ liệu người dùng của các hệ thống mạng xã hội hiện nay, thông qua việc ứng dụng mô hình cơ sở dữ liệu hiện đại và linh hoạt.

Hệ thống mạng xã hội phải xử lý khối lượng dữ liệu rất lớn và đa dạng, bao gồm thông tin hồ sơ người dùng, bài đăng, bình luận, lượt thích, tin nhắn, thông báo, dữ liệu media, báo cáo vi phạm, theo dõi và mối quan hệ giữa người dùng. Dữ liệu được tạo ra liên tục và thường xuyên thay đổi theo thời gian, đòi hỏi hệ thống phải có mô hình tổ chức quản lý phù hợp.

Phần lớn các hệ thống mạng xã hội ban đầu hoặc các hệ thống quản lý dữ liệu truyền thống được xây dựng dựa trên mô hình cơ sở dữ liệu quan hệ SQL. Mô hình này đảm bảo tốt tính toàn vẹn dữ liệu, tuy nhiên lại gặp nhiều hạn chế khi áp dụng cho môi trường mạng xã hội có dữ liệu lớn và biến động nhanh, cụ thể như:

- Thiếu tính linh hoạt: Lọc đồ dữ liệu cố định gây khó khăn khi cần bổ sung hoặc thay đổi cấu trúc dữ liệu người dùng và các loại nội dung phát sinh mới.

- Hạn chế mở rộng: Khi số lượng người dùng và dữ liệu tăng nhanh, việc mở rộng cơ sở dữ liệu quan hệ trở nên phức tạp và kém hiệu quả.

- Tốc độ truy vấn: Các thao tác truy xuất dữ liệu người dùng và mối quan hệ thường yêu cầu nhiều phép nối bảng, làm giảm hiệu suất hệ thống.

Để giải quyết các vấn đề trên, đề tài đề xuất chuyển đổi sang mô hình cơ sở dữ liệu NoSQL, sử dụng hệ quản trị cơ sở dữ liệu MongoDB để lưu trữ dữ liệu sẽ giúp cho việc quản lý dữ liệu trở nên trực quan hơn:

- Tăng cường tính linh hoạt để dễ dàng thích nghi với sự thay đổi liên tục của dữ liệu người dùng.

- Tối ưu hóa tốc độ truy xuất thông qua các mô hình thiết kế lược đồ phù hợp với các nghiệp vụ truy cập thường xuyên.

- Khả năng mở rộng quy mô nhanh, đáp ứng tốt nhu cầu phát triển và gia tăng dữ liệu của hệ thống mạng xã hội trong tương lai.

CHƯƠNG 2: NGHIÊN CỨU LÝ THUYẾT

2.1. Giới thiệu về NoSQL

2.1.1. Khái niệm

Cơ sở dữ liệu NoSQL (Not only SQL hoặc Not SQL) là một hệ thống quản lý dữ liệu không quan hệ -non relational Data Management System sở hữu lược đồ linh hoạt. Đặc trưng của cơ sở dữ liệu NoSQL là có thể dễ dàng mở rộng khi cần thiết. Vì vậy, việc sử dụng thường được ứng dụng cho những kho dữ liệu phân tán với khả năng lưu trữ lượng dữ liệu lớn, hay những ứng dụng web dữ trên thời gian thực.

Xuất hiện cuối thế cuối thế kỉ XX, NoSQL đã mở ra những hướng tiếp cận mới cho việc lưu trữ dữ liệu. Dữ liệu phức tạp với nhiều kích cỡ và hình dạng khác nhau đều có thể được lưu trữ trong NoSQL. Từ dữ liệu có cấu trúc đến dữ liệu bán cấu trúc và phi cấu trúc. Trải qua nhiều năm phát triển, NoSQL đã được sử dụng trong nhiều lĩnh vực khác nhau. Nhiều doanh nghiệp và tổ chức hiện đang sử dụng NoSQL để mở rộng việc phân phối dữ liệu của họ trên đám mây.

2.1.2. Các kiểu mô hình dữ liệu trong NoSQL

2.1.2.1. Key-Value Store

- Cơ sở dữ liệu dạng Key-Value Stores có các đặc điểm như sau: có một tập các đối tượng dữ liệu nhận dạng, các khóa; đối với mỗi khóa, có chính xác một đối tượng dữ liệu mô tả có liên quan, giá trị cho khóa đó; chỉ định một khóa cho phép truy vấn giá trị liên quan trong cơ sở dữ liệu.

Để truy vấn dữ liệu trong CSDL, ta dựa vào key để lấy dữ liệu ra. Các CSDL dạng này có tốc độ truy vấn rất nhanh.

- Database tiêu biểu: Riak, Redis, MemCache, Project Voldemort, CouchBase

- Ứng dụng: Do tốc độ truy xuất nhanh, key-value stores thường được dùng để làm cache cho ứng dụng. Ngoài ra, nó còn được dùng để lưu thông tin trong sessions, profiles/preferences của user.

2.1.2.2. Document Store

- Mỗi đối tượng sẽ được lưu trữ trong CSDL dưới dạng một document. Dữ liệu thường được biểu diễn theo các định dạng như BSON, JSON hoặc XML và không

bị ràng buộc bởi lược đồ cố định như trong SQL. Do đó ta có thể thêm/sửa các thuộc tính, thay đổi table, rất nhanh và đơn giản. Cơ sở dữ liệu dạng này có tốc độ truy vấn nhanh, có thể thực hiện các câu truy vấn phức tạp, dễ mở rộng. Mỗi cơ sở dữ liệu có một kiểu truy vấn riêng.

- Một số cơ sở dữ liệu tiêu biểu thuộc nhóm Document Database gồm: MongoDB, RavenDB, CouchDB, TerraStone và OrientDB.

- Ứng dụng: Do nhanh và linh động, document database thường đóng vai trò làm database cho các ứng dụng prototype, big data, e-commerce, CMS. Ngoài ra, ta còn dùng nó để lưu log hoặc history.

2.1.2.3. Column-family Store

- Column-Family Database lưu trữ dữ liệu theo cột thay vì theo hàng như SQL, mỗi hàng có một key/id riêng và số lượng cột giữa các hàng có thể khác nhau. Loại cơ sở dữ liệu này có cú pháp truy vấn tương đối giống SQL.

- Database tiêu biểu: Cassandra (Phát triển bởi Facebook), HyperTable, Apache Hbase.

- Ứng dụng: Column-Family Database được sử dụng khi ta cần ghi một số lượng lớn dữ liệu, big data. Nó còn được ứng dụng trong 1 số CMS và ứng dụng ecommerce.

2.1.2.4. Graph Database

- Dữ liệu trong graph database được lưu dưới dạng các node. Mỗi node sẽ có 1 label, 1 số properties như một row trong SQL. Các node này được kết nối với nhau bằng các relationship. Graph database tập trung nhiều vào relationship giữa các node, áp dụng nhiều thuật toán duyệt node để tăng tốc độ.

- Database tiêu biểu: Neo4j, InfiniteGraph, OrientDB, HYPERGRAPHDB

- Ứng dụng: Graph Database đặc biệt phù hợp cho các bài toán truy vấn quan hệ như mạng xã hội, hệ thống gợi ý, giao dịch tài chính hoặc mạng nơ-ron. Neo4j là một hệ cơ sở dữ liệu miễn phí và có cộng đồng hỗ trợ lớn, thuận lợi cho việc học tập và triển khai.

2.1.3. Các khái niệm trong cơ sở dữ liệu NoSQL

2.1.3.1. Schema-less / Flexible Schema (Lược đồ linh hoạt)

NoSQL không yêu cầu cấu trúc dữ liệu cố định. Các bản ghi trong cùng một collection có thể khác nhau về trường và kiểu dữ liệu, giúp hệ thống linh hoạt và dễ mở rộng khi thay đổi yêu cầu nghiệp vụ.

2.1.3.2. Document Database

Dữ liệu được lưu dưới dạng các document (JSON/BSON/XML), mỗi document đại diện cho một đối tượng. Mô hình này hỗ trợ truy vấn linh hoạt, dễ mở rộng và phù hợp với các ứng dụng web, CMS, thương mại điện tử.

2.1.3.3. Collection

Collection là tập hợp các document trong cơ sở dữ liệu NoSQL, tương tự như bảng (table) trong hệ cơ sở dữ liệu quan hệ. Tuy nhiên, các document trong cùng một collection không bắt buộc phải có cấu trúc giống nhau, cho phép lưu trữ dữ liệu linh hoạt và dễ mở rộng.

2.1.3.4. Key-Value Store

Dữ liệu được tổ chức dưới dạng cặp khóa–giá trị, truy vấn dựa trên key. Nhờ cấu trúc đơn giản, loại cơ sở dữ liệu này có tốc độ truy xuất rất nhanh

2.1.3.5. Replication

Dữ liệu được sao chép giữa các máy chủ nhằm đảm bảo an toàn, tăng tính sẵn sàng và cải thiện khả năng xử lý truy vấn đọc.

2.1.3.6. Định lý CAP

Một hệ thống phân tán không thể đồng thời đảm bảo tính nhất quán, tính sẵn sàng và khả năng chịu lỗi phân vùng. NoSQL thường ưu tiên khả năng chịu lỗi phân vùng và đánh đổi giữa tính nhất quán và tính sẵn sàng.

2.1.3.7. Mô hình BASE

BASE tập trung vào tính sẵn sàng cao, trạng thái mềm và tính nhất quán cuối cùng, cho phép dữ liệu được đồng bộ sau một khoảng thời gian để tối ưu hiệu năng và khả năng mở rộng.

2.1.4. Một số ưu điểm và nhược điểm của cơ sở dữ liệu NoSQL

2.1.4.1. Ưu điểm:

- NoSQL phù hợp để xử lý các bài toán dữ liệu lớn trong những hệ thống thông tin phân tán.
- NoSQL hỗ trợ cơ chế sao chép dữ liệu tự động, giúp đảm bảo an toàn và khôi phục dữ liệu khi xảy ra sự cố.
- Hệ thống có khả năng mở rộng linh hoạt bằng cách bổ sung nhiều máy chủ để phân tán tải, thay vì nâng cấp một máy chủ duy nhất.

2.1.4.2. Nhược điểm:

- Quản lý dữ liệu: Việc quản lý dữ liệu trong NoSQL phức tạp hơn so với cơ sở dữ liệu quan hệ, quá trình cài đặt và vận hành hằng ngày thường tốn nhiều thời gian.
- Sao lưu dữ liệu: Sao lưu dữ liệu chưa được hỗ trợ nhất quán ở một số hệ cơ sở dữ liệu NoSQL, gây khó khăn trong việc đảm bảo an toàn dữ liệu.
- Thiếu tính nhất quán: NoSQL thường đánh đổi tính nhất quán để ưu tiên hiệu năng và tốc độ xử lý.
- Trọng tâm hẹp: Cơ sở dữ liệu NoSQL có phạm vi chức năng hẹp, chủ yếu tập trung vào lưu trữ dữ liệu.
- Mã nguồn mở: Phần lớn NoSQL là mã nguồn mở và chưa có tiêu chuẩn chung, dẫn đến sự khác biệt giữa các hệ quản trị.
- Không có lược đồ: Do không có lược đồ cố định, trách nhiệm kiểm soát cấu trúc và ràng buộc dữ liệu được chuyển sang phía nhà phát triển, lập trình ứng dụng.
- Kỹ năng NoSQL: NoSQL là công nghệ tương đối mới, do đó yêu cầu người sử dụng có kỹ năng và kinh nghiệm chuyên môn phù hợp.

2.2. Giới thiệu về MongoDB

2.2.1. Tổng quan về MongoDB

- MongoDB là một cơ sở dữ liệu đa nền tảng, hoạt động trên các khái niệm Collection và Document, nó cung cấp hiệu suất cao, tính khả dụng cao và khả năng mở rộng dễ dàng.
- MongoDB có lược đồ linh hoạt, cho phép các document trong cùng một collection có cấu trúc khác nhau.

- Dữ liệu được tổ chức rõ ràng dưới dạng document, thuận tiện cho việc phát triển ứng dụng.
- Không cần sử dụng các phép join phức tạp như trong RDBMS.
- Hỗ trợ truy vấn linh hoạt và truy vấn sâu trên document bằng ngôn ngữ truy vấn mạnh.

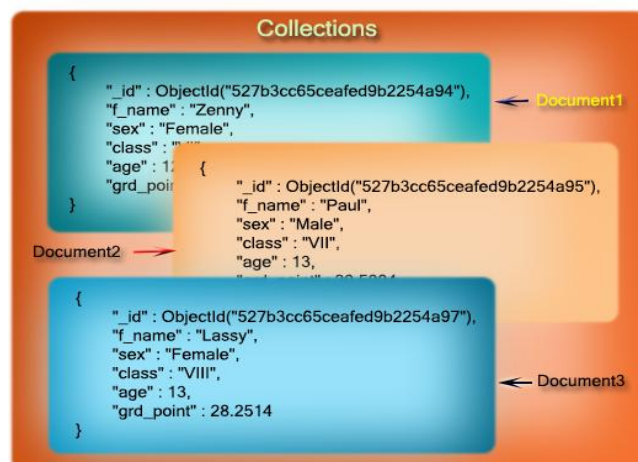
2.2.2. Một số khái niệm cơ bản trong MongoDB

- Khái niệm Database: Database là một nơi chứa vật lý cho các Collection.

Mỗi Database lấy tập hợp các file riêng của nó trên hệ thống file. Mỗi MongoDB Server có thể có nhiều cơ sở dữ liệu.

- Khái niệm Collection: Collection là một nhóm các Document trong MongoDB. Collection tương đương như một Table trong RDBMS. Do đó một Collection tồn tại bên trong một cơ sở dữ liệu duy nhất. Các Collection không có ràng buộc như các hệ quản trị cơ sở dữ liệu quan hệ nên việc truy xuất rất nhanh. Do đó mỗi collection có thể chứa nhiều loại khác nhau, không giống như table trong hệ quản trị cơ sở dữ liệu quan hệ có các field cố định. Các Document bên trong một Collection có thể có nhiều trường khác nhau. Đặc biệt, tất cả các Document trong một Collection là tương tự nhau hoặc với cùng mục đích liên quan.

- Khái niệm Document: Một Document trong MongoDB có cấu trúc tương tự như kiểu dữ liệu JSON là một tập hợp các cặp key-value. Các Document có lược đồ động, nghĩa là Document trong cùng một Collection không cần thiết phải có cùng một tập hợp các trường hoặc cấu trúc giống nhau, và các trường chung trong Document của một Collection có thể giữ các kiểu dữ liệu khác nhau.



Hình 2.1: Minh họa Document trong Collection

Bảng dưới đây trình bày mối quan hệ giữa các thuật ngữ của hệ quản trị cơ sở dữ liệu quan hệ và MongoDB.

Bảng 2.1: Mối quan hệ giữa các thuật ngữ của RDBMS và MongoDB

RDBMS	MongoDB
Database	Database
Table	Collection
Tuple/Row colum	Document Field
Table Join	Embedded Document
Primary Key	Primary Key(Giá trị mặc định là _id được cung cấp bởi chính MongoDB)

2.2.3. Các kiểu dữ liệu trong MongoDB

- Chuỗi: Đây là kiểu dữ liệu được sử dụng phổ biến nhất để lưu giữ dữ liệu. Chuỗi trong MongoDB phải là UTF-8 hợp lệ.
- Số nguyên: Kiểu dữ liệu này được sử dụng để lưu một giá trị số. Số nguyên có thể là 32 bit hoặc 64 bit phụ thuộc vào Server của bạn.
- Boolean: Kiểu dữ liệu này được sử dụng để lưu giữ một giá trị Boolean (true/false).
- Double: Kiểu dữ liệu này được sử dụng để lưu các giá trị số thực dấu chấm động.
- Min/ Max keys: Kiểu dữ liệu này được sử dụng để so sánh một giá trị với các phần tử BSON thấp nhất và cao nhất.
- Mảng: Kiểu dữ liệu này được sử dụng để lưu giữ các mảng hoặc danh sách hoặc nhiều giá trị vào trong một key.
- Timestamp: Giúp thuận tiện cho việc ghi chép hoặc đánh dấu thời điểm một Document được sửa đổi hoặc được thêm vào.
- Object: Kiểu dữ liệu này được sử dụng cho các Document được nhúng vào.
- Null: Kiểu dữ liệu này được sử dụng để lưu một giá trị Null.
- Symbol: Kiểu dữ liệu này được sử dụng giống như một chuỗi, tuy nhiên, nói chung nó được dành riêng cho các ngôn ngữ mà sử dụng kiểu symbol cụ thể.

-
- Date: Kiểu dữ liệu này được sử dụng để lưu giữ date và time hiện tại trong định dạng UNIX time. Bạn có thể xác định date time riêng cho bạn bằng việc tạo đối tượng Date và truyền ngày, tháng, năm vào trong đó.
 - Object ID: Kiểu dữ liệu này được sử dụng để lưu giữ ID của Document.
 - Binary data: Kiểu dữ liệu này được sử dụng để lưu giữ dữ liệu nhị phân.
 - Code: Kiểu dữ liệu này được sử dụng để lưu giữ JavaScript code vào trong Document.
 - Regular expression: Kiểu dữ liệu này được sử dụng để lưu giữ Regular Expression

2.2.4. Các thao tác dữ liệu trên MongoDB

2.2.4.1. Tạo cơ sở dữ liệu

- Sử dụng lệnh use để tạo cơ sở dữ liệu, các cơ sở dữ liệu được xác định bởi tên của nó. Lệnh use sẽ tạo một cơ sở dữ liệu mới nếu cơ sở dữ liệu chưa tồn tại, nếu cơ sở dữ liệu đã tồn tại thì kết quả sẽ trả về cơ sở dữ liệu đang tồn tại.
- Cú pháp cơ bản: use <DATABASE_NAME>
- Trong đó quy tắc đặt tên cho < DATABASE_NAME > như sau
 - + Gồm chữ và số;
 - + Phân biệt chữ hoa hay chữ thường;
 - + Không chứa chuỗi rỗng (“”);
 - + Không chứa các ký tự sau: /, \, ., ", *, , , |, ?, \$, (khoảng trắng), or \0 (the null character)
 - + Tên cơ sở dữ liệu giới hạn tối đa 64 bytes.

2.2.4.2. Kiểm tra cơ sở dữ liệu hiện hành

Sử dụng lệnh db với cú pháp như sau:

```
>db  
<MyNoSQL
```

2.2.4.3. Kiểm tra danh sách các cơ sở dữ liệu

Sử dụng lệnh show dbs.

```
>show dbs
```

- Cơ sở dữ liệu mặc định trong MongoDB là test, nếu người dùng không tạo thêm bất kỳ cơ sở dữ liệu nào thì các Collection sẽ được lưu trữ trong test.

- Khi một cơ sở dữ liệu được tạo mà chưa có bất kỳ Collection nào thì tên cơ sở dữ liệu đó sẽ không hiện lên trong danh sách các cơ sở dữ liệu khi sử dụng lệnh show dbs.

2.2.4.4. Xóa cơ sở dữ liệu

Sử dụng lệnh db.dropDatabase()

Lệnh này sẽ xóa các cơ sở dữ liệu đã chọn. Nếu không chọn cơ sở dữ liệu nào thì sẽ xóa cơ sở dữ liệu mặc định test.

Trước khi xóa thì dùng lệnh show dbs để kiểm tra danh sách các cơ sở dữ liệu hiện có sau đó chọn cơ sở dữ liệu cần xóa.

>show dbs (hiển thị danh sách các db hiện có)

>use MyNoSQL

< 'switched to db MyNoSQL'

>db.dropDatabase()

>{ "dropped" : "Tên cơ sở dữ liệu cần xóa", "ok" : 1 }

Kiểm tra lại xem cơ sở dữ liệu đã xóa chưa

>show dbs

(Hiển thị danh sách các db hiện có, kiểm tra xem db đã xóa còn tồn tại không)

2.2.4.5. Tạo Collection

- Sử dụng phương thức db.createCollection(name, options) để tạo Collection.

- Trong đó name là tên của Collection. Options là một Document và được sử dụng để xác định cấu hình cho Collection.

Bảng 2.2: Tham số options của Collection

Tham số	Kiểu	Miêu tả
Name	Chuỗi	Tên của Collection
Options	Document	(Tùy chọn) Xác định

Tham số options là tùy chọn, người dùng cần xác định tên của Collection.

Dưới đây là danh sách các tùy chọn cho options người dùng có thể sử dụng:

Bảng 2.3. Danh sách các tùy chọn options của Collection

Trường	Kiểu	Miêu tả
capped	Boolean	(Tùy chọn) Nếu true, kích hoạt một Capped Collection. Đây là một Collection có kích thước cố định mà tự động ghi đè các entry cũ nhất khi nó tiếp cận kích cỡ tối đa. Nếu xác định là true, thì cần xác định tham số size>
autoIndexID	Boolean	(Tùy chọn) Nếu true, tự động tạo chỉ mục trên các trường _id. Giá trị mặc định là false.
Size	Số	(Tùy chọn) Xác định kích thước tối đa (giá trị byte) cho một Capped Collection. Nếu tham số capped là true, thì cần xác định trường này.
Max	Số	(Tùy chọn) Xác định số Document tối đa được cho phép trong một Capped Collection.

Trong khi thực hiện việc chèn dữ liệu vào Document, đầu tiên MongoDB kiểm tra trường size của Capped Collection, sau đó nó kiểm tra trường max.

2.2.4.6. Xóa Collection

Phương thức `db.collection.drop()` trong MongoDB được sử dụng để xóa một Collection từ cơ sở dữ liệu.

2.2.4.7. Thêm mới Document

Sử dụng phương thức `insert()` hoặc `save()` để thêm Document vào Collection
Cú pháp cơ bản của lệnh `insert()` như sau:

```
db.COLLECTION_NAME.insert(document)
```

2.2.4.8. Truy vấn Document

Để truy vấn dữ liệu từ Collection trong MongoDB, bạn cần sử dụng phương thức `find()` trong MongoDB.

Cú pháp cơ bản của phương thức `find()` là như sau:

```
db.COLLECTION_NAME.find()  
db.COLLECTION_NAME.findOne()
```

2.2.4.9. Cập nhật Document

Phương thức `update()` hoặc `save()` trong MongoDB được sử dụng để cập nhật Document vào trong một Collection. Phương thức `update()` cập nhật các giá trị

trong Document đang tồn tại trong khi phương thức save() thay thế Document đang tồn tại với Document đã truyền trong phương thức save() đó.

Phương thức update() cập nhật các giá trị trong Document đang tồn tại.

Cú pháp cơ bản của phương thức update() là như sau:

```
>db.<collectionname>.update(SELECTION_CRITERIA,UPDATED_DATA)
```

2.2.4.10. Xóa Document trong MongoDB

- Phương thức remove() được sử dụng để xóa Document từ Collection.

- Phương thức remove() nhận hai tham số. Tham số đầu tiên deletion criteria xác định Document để xóa, và tham số thứ hai là justOne.

1. deletion criteria: Xác định Document để xóa.

2. justOne : Nếu được thiết lập là true (hoặc 1), thì chỉ xóa một Document.

Cú pháp cơ bản của phương thức remove() là như sau:

```
>db.COLLECTION_NAME.remove(DELETION_CRITTERIA)
```

2.2.4.11. Tạo backup

Để tạo Backup của cơ sở dữ liệu trong MongoDB, sử dụng lệnh mongodump. Lệnh này sẽ dump tất cả dữ liệu của Server vào trong dump directory. Có nhiều tùy chọn có sẵn từ đó người dùng có thể giới hạn lượng dữ liệu hoặc tạo backup của Remote Server.

Cú pháp cơ bản của lệnh mongodump trong MongoDB là:

```
>mongodump
```

Bảng 2.4: Các lệnh tùy chọn trong lệnh Backup

Cú pháp	Miêu tả	Ví dụ
mongodump --host HOST_NAME --port PORT_NUMBER	Lệnh này sẽ sao lưu tất cả cơ sở dữ liệu của mongod instance đã xác định	mongodump -- host tutorialspoint.com --port 27017
mongodump --dbpath DB_PATH --out BACKUP_DIRECTORY	Lệnh này chỉ sao lưu cơ sở dữ liệu đã xác định tại path đã cho.	mongodump -- dbpath /data/db/ - -out /data/backup/
mongodump --collection	Lệnh này chỉ sao lưu	mongodump --

COLLECTION --db DB_NAME	Collection đã xác định của cơ sở dữ liệu đã cho	collection mycol - -db test
----------------------------	--	--------------------------------

2.2.4.12. Phục hồi dữ liệu trong MongoDB

Để phục hồi dữ liệu đã sao lưu trong MongoDB, bạn sử dụng lệnh `mongorestore`. Lệnh này phục hồi tất cả dữ liệu từ thư mục sao lưu.

Cú pháp cơ bản của lệnh `mongorestore` là:

```
>mongorestore
```

2.2.5. Cơ chế Index và Aggregation

Cơ chế Index (Đánh chỉ mục)

Index là một cấu trúc dữ liệu đặc biệt giúp MongoDB truy xuất dữ liệu một cách hiệu quả. Nếu không có Index, MongoDB phải thực hiện quét toàn bộ bộ sưu tập (Collection Scan) để tìm các tài liệu phù hợp, điều này gây tốn tài nguyên và kéo dài thời gian xử lý khi tập dữ liệu lớn.

Mục đích: Giảm thiểu số lượng tài liệu cần phải kiểm tra khi thực hiện truy vấn, từ đó tối ưu hóa tốc độ tìm kiếm và các thao tác sắp xếp dữ liệu.

Cấu trúc: Theo mặc định, MongoDB tự động tạo một chỉ mục duy nhất trên trường `_id` cho mọi bộ sưu tập để định danh tài liệu. Để nâng cao hiệu năng thực tế, người dùng có thể chủ động tạo thêm các chỉ mục trên các trường thường xuyên được sử dụng trong bộ lọc tra cứu.

Cơ chế Aggregation (Xử lý dữ liệu tập trung)

Aggregation là một khung xử lý (Framework) mạnh mẽ cho phép thực hiện các phép toán thống kê và biến đổi dữ liệu thông qua quy trình đường ống (Pipeline).

Định nghĩa: Thay vì chỉ trả về các tài liệu thô, Aggregation tiến hành xử lý các nhóm dữ liệu để trả về kết quả đã được tính toán như tổng (`$sum`), trung bình (`$avg`), hay đếm (`$count`).

Aggregation Pipeline: Đây là cơ chế cốt lõi trong xử lý dữ liệu của MongoDB, nơi dữ liệu đi qua một chuỗi các giai đoạn xử lý (Stages). Kết quả đầu ra của giai đoạn này sẽ trở thành đầu vào của giai đoạn kế tiếp:

\$match: Thực hiện lọc các tài liệu thỏa mãn điều kiện cụ thể, tương đương với mệnh đề WHERE trong SQL truyền thống.

\$group: Gom nhóm các tài liệu theo một khóa xác định (ví dụ: mã khoa, mã lớp) để thực hiện các phép toán thống kê.

\$merge: Đây là giai đoạn quan trọng giúp ghi kết quả thống kê vào một bộ sưu tập mới. Cơ chế này giúp các báo cáo luôn sẵn sàng để truy xuất mà không cần phải thực hiện tính toán lại từ đầu trên toàn bộ dữ liệu gốc.

CHƯƠNG 3: HIỆN THỰC HÓA NGHIÊN CỨU

3.1. Cài đặt và chuẩn bị môi trường

3.1.1. Hạ tầng phần mềm

Hệ điều hành: Windows 11 64-bit.

Hệ quản trị cơ sở dữ liệu: MongoDB Community Server phiên bản 6.0.

Phiên bản này hỗ trợ mạnh mẽ các toán tử truy vấn trên mảng và document nhúng.

3.1.2. Công cụ quản trị và thao tác

MongoDB Compass: công cụ giao diện đồ họa được sử dụng để thiết kế dữ liệu, nhập liệu mẫu và chỉnh sửa document.

MongoDB Shell: công cụ dòng lệnh được sử dụng để khởi tạo database và collection, thực thi các câu lệnh truy vấn phức tạp.

3.2. Lược đồ tổng quát cơ sở dữ liệu

Bảng 3. 1: Lược đồ cơ sở dữ liệu

Collection	Thuộc tính cấp 1	Thuộc tính cấp 2
nguoi_dung	1. Id	
	2. Ten_dang_nhap	
	3. mat_khau	
	4. email	
	5. ho_ten	
	6. anh_dai_dien	
	7. tieu_su	
	8. ngay_sinh	
	9. ngay_tao	
	10. ngay_cap_nhat	
	11. so_nguoi_theo_doi	
	12. so_dang_theo_do	
bai_dang	1. _id	
	2. nguoi_dung_id	
	3. tieu_de	
	4. noi_dung	

Collection	Thuộc tính cấp 1	Thuộc tính cấp 2
	5. tep_phuong_tien	
	6. ngay_tao	
	7. ngay_cap_nhat	
	8. so_luot_thich	
	9. so_binh_luan	
binh_luan	1. _id	
	2. bai_dang_id	
	3. nguoi_dung_id	
	4. noi_dung	
	5. ngay_tao	
	6. so_luot_thich	
	7. phan_hoi	7.1. nguoi_dung_id
		7.2. noi_dung
		7.3. ngay_tao
luot_thich	1. _id	
	2. nguoi_dung_id	
	3. bai_dang_id	
	4. binh_luan_id	
	5. ngay_tao	
tin_nhan	1. _id	
	2. nguoi_gui_id	
	3. nguoi_nhan_id	
	4. noi_dung	
	5. tep_dinh_kem	
	6. ngay_tao	
thong_bao	1. _id	
	2. nguoi_nhan_id	
	3. nguoi_gui_id	
	4. loai	

Collection	Thuộc tính cấp 1	Thuộc tính cấp 2
	5. bai_dang_id	
	6. binh_luan_id	
	7. noi_dung	
	8. da_doc	
	9. ngay_tao	
theo_doi	1. _id	
	2. nguoi_theo_doi_id	
	3. nguoi_duoc_theo_doi_id	
	4. ngay_tao	
media	1. _id	
	2. bai_dang_id	
	3. nguoi_dung_id	
	4. loai	
	5. url	
	6. ngay_tao	
bao_cao	1. _id	
	2. nguoi_bao_cao	
	3. doi_tuong_bi_bao_cao	
	4. loai	
	5. ly_do	
	6. ngay_tao	

3.3. Hiện thực hóa cấu trúc dữ liệu

3.3.1. Chiến lược thiết kế

Mục tiêu: Tối ưu hóa hiệu suất cho các thao tác đọc dữ liệu trong hệ thống người dùng mạng xã hội.

Giải pháp: Áp dụng mô hình nhúng trong NoSQL, trong đó các thông tin liên quan đến người dùng như hồ sơ cá nhân, danh sách bạn bè, theo dõi và cài đặt cơ bản được lưu trực tiếp trong document người dùng. Cách thiết kế này giúp hiển thị

đầy đủ thông tin chỉ với một lần truy vấn, giảm độ trễ và nâng cao hiệu suất hệ thống.

3.3.2. Chi tiết cấu trúc Document của các Collection

3.3.2.1. Collection `nguoi_dung` (Collection Trung tâm)

Collection `nguoi_dung` là collection trung tâm của hệ thống mạng xã hội, lưu trữ hồ sơ cơ bản của người dùng. Các collection khác như bài đăng, bình luận, lượt thích, theo dõi, tin nhắn, thông báo, media và báo cáo vi phạm đều liên kết với `nguoi_dung` thông qua khóa tham chiếu `ObjectId`. Document được thiết kế theo dạng không nhúng, phù hợp với các thông tin ổn định và thường xuyên được tham chiếu trong hệ thống.

* Cấu trúc collection `nguoi_dung`

```
{
  "_id": "ObjectId",           // Id người dùng
  "ten_dang_nhap": "String",    // Tên đăng nhập
  "mat_khau": "String",        // Mật khẩu (đã mã hóa)
  "email": "String",           // Email
  "ho_ten": "String",          // Họ tên
  "anh_dai_dien": "String",    // Ảnh đại diện
  "tieu_su": "String",         // Tiểu sử
  "ngay_sinh": "Date",         // Ngày sinh
  "ngay_tao": "Date",          // Ngày tạo tài khoản
  "ngay_cap_nhat": "Date",     // Ngày cập nhật
  "so_nguoi_theo_doi": "Number", // Số người theo dõi
  "so_dang_theo_doi": "Number" // Số đang theo dõi
}
```

3.3.2.2. Collection `bai_dang`

Collection `bai_dang` dùng để lưu trữ các bài đăng của người dùng trong hệ thống mạng xã hội. Mỗi bài đăng tham chiếu đến người dùng đăng bài thông qua `Id` người đăng, đảm bảo liên kết dữ liệu giữa các collection. Các trường số lượt thích

và số bình luận được lưu dưới dạng dữ liệu tổng hợp nhằm tối ưu hiệu suất cho các thao tác đọc dữ liệu.

*** Cấu trúc collection `bai_dang`**

```
{
  "_id": { "$oid": "ObjectId" },           // Id bài đăng
  "nguoi_dung_id": { "$oid": "ObjectId" }, // Id người đăng (id người dùng)
  "tieu_de": "String",                     // Tiêu đề bài đăng
  "noi_dung": "String",                     // Nội dung bài đăng
  "tep_phuong_tien":                       // Tập phương tiện (kiểu Array)
    [ "String" ],                          // Đường dẫn ảnh, video, file,...
  "ngay_tao": { "$date": "Date" },         // Ngày tạo
  "ngay_cap_nhat": { "$date": "Date" },    // Ngày cập nhật
  "so_luot_thich": "Number",               // Số lượt thích
  "so_binh_luan": "Number"                 // Số bình luận
}
```

3.3.2.3. Collection `binh_luan`

Collection `binh_luan` dùng để lưu trữ các bình luận của người dùng trên bài đăng trong hệ thống mạng xã hội. Mỗi bình luận tham chiếu đến bài đăng và người bình luận thông qua các khóa `ObjectId`. Các phản hồi cho bình luận được lưu dưới dạng mảng document nhúng, giúp quản lý và hiển thị nhanh các luồng thảo luận liên quan.

*** Cấu trúc collection `binh_luan`**

```
{
  "_id": "ObjectId",                       // Id bình luận
  "bai_dang_id": "ObjectId",               // Id bài đăng (tham chiếu bài đăng)
  "nguoi_dung_id": "ObjectId",             // Id người bình luận (tham chiếu người dùng)
  "noi_dung": "String",                     // Nội dung bình luận
  "ngay_tao": "Date",                       // Ngày tạo
  "so_luot_thich": "Number",               // Số lượt thích
  "phan_hoi": [                             // Phản hồi (Embedded Document Array)
```



```
{
  "nguoi_dung_id": "ObjectId", // Id người dùng trả lời
  "noi_dung": "String",        // Nội dung phản hồi
  "ngay_tao": "Date"           // Ngày tạo phản hồi
}
]
```

3.3.2.4. Collection **luot_thich**

Collection **luot_thich** dùng để ghi nhận hành động thích của người dùng đối với bài đăng hoặc bình luận trong hệ thống mạng xã hội. Mỗi lượt thích tham chiếu đến người dùng và đối tượng được thích, phục vụ việc thống kê và quản lý tương tác.

* Cấu trúc collection **luot_thich**

```
{
  "_id": "ObjectId", // Id lượt thích
  "nguoi_dung_id": "ObjectId", // Id người dùng (tham chiếu người dùng)
  "bai_dang_id": "ObjectId", // Id bài đăng (tham chiếu bài đăng)
  "binh_luan_id": "ObjectId", // Id bình luận (tham chiếu bình luận)
  "ngay_tao": "Date" // Ngày tạo
}
```

3.3.2.5. Collection **tin_nhan**

Collection **tin_nhan** dùng để lưu trữ các tin nhắn trao đổi giữa người dùng trong hệ thống mạng xã hội. Mỗi tin nhắn tham chiếu đến người gửi và người nhận, hỗ trợ gửi nội dung và tệp đính kèm.

* Cấu trúc collection **tin_nhan**

```
{
  "_id": "ObjectId", // Id tin nhắn
  "nguoi_gui_id": "ObjectId", // Id người gửi
  "nguoi_nhan_id": "ObjectId", // Id người nhận
  "noi_dung": "String", // Nội dung tin nhắn
}
```

```
"tep_dinh_kem": "String",           // Tập đính kèm
"ngay_tao": "Date"                  // Ngày tạo
}
```

3.3.2.6. Collection `thong_bao`

Collection `thong_bao` dùng để lưu trữ các thông báo gửi đến người dùng, phát sinh từ các hoạt động như thích bài đăng, bình luận, theo dõi hoặc tin nhắn. Mỗi thông báo tham chiếu đến người nhận, người tạo thông báo và đối tượng liên quan, đồng thời lưu trạng thái đã đọc.

* Cấu trúc collection `thong_bao`

```
{
  "_id": "ObjectId",                // Id thông báo
  "nguoi_nhan_id": "ObjectId",       // Id người nhận thông báo
  "nguoi_gui_id": "ObjectId",        // Id người tạo/gửi thông báo
  "loai": "String",                  // (like_post, comment, follow, message)
  "bai_dang_id": "ObjectId",         // Id bài đăng (nếu có)
  "noi_dung": "String",              // Nội dung thông báo
  "da_doc": "Boolean",               // Trạng thái đã đọc
  "ngay_tao": "Date"                 // Ngày tạo
}
```

3.3.2.7. Collection `theo_doi`

Collection `theo_doi` dùng để quản lý mối quan hệ theo dõi giữa các người dùng trong hệ thống. Mỗi bản ghi thể hiện một hành động theo dõi của người dùng đối với người dùng khác.

* Cấu trúc collection `theo_doi`

```
{
  "_id": "ObjectId",                // Id theo dõi
  "nguoi_theo_doi_id": "ObjectId",  // Id người theo dõi
  "nguoi_duoc_theo_doi_id": "ObjectId", // Id người được theo dõi
  "ngay_tao": "Date"                 // Ngày tạo
}
```

3.3.2.8. Collection media

Collection media dùng để lưu trữ thông tin các tệp phương tiện (ảnh, video, file) được người dùng tải lên hệ thống. Dữ liệu trong collection này phục vụ cho việc liên kết với bài đăng, tin nhắn và các chức năng khác của mạng xã hội.

* Cấu trúc collection media

```
{
  "_id": "ObjectId",           // Id media
  "bai_dang_id": "ObjectId",   // Tham chiếu bài đăng
  "nguoi_dung_id": "ObjectId", // Người tải lên
  "loai": "String",           // Loại media (image, video, file)
  "url": "String",            // Đường dẫn file
  "ngay_tao": "Date"          // Ngày tạo
}
```

3.3.2.9. Collection bao_cao

Collection bao_cao dùng để lưu trữ các báo cáo vi phạm do người dùng gửi, áp dụng cho bài đăng, bình luận hoặc tài khoản người dùng.

* Cấu trúc collection bao_cao

```
{
  "_id": "ObjectId",           // Id báo cáo
  "nguoi_bao_cao_id": "ObjectId", // Người báo cáo
  "doi_tuong_id": "ObjectId",   // Đối tượng bị báo cáo
  "loai": "String",            // Loại (post, comment, user)
  "ly_do": "String",           // Lý do báo cáo
  "ngay_tao": "Date"          // Ngày tạo
}
```

3.4 Khởi tạo và cấu hình cơ sở dữ liệu

Thay vì để MongoDB tự động tạo Collection khi dữ liệu được thêm vào, việc chủ động khai báo Collection ngay từ đầu giúp hệ thống có cấu trúc rõ ràng, dễ quản lý hơn, đồng thời giảm thiểu rủi ro tạo nhầm Collection do lỗi chính tả trong quá trình nhập liệu.

```
> use mangxh
< switched to db mangxh
> db.createCollection("nguoi_dung")

db.createCollection("bai_dang")

db.createCollection("binh_luan")

db.createCollection("luot_thich")

db.createCollection("tin_nhan")

db.createCollection("media")

db.createCollection("bao_cao")

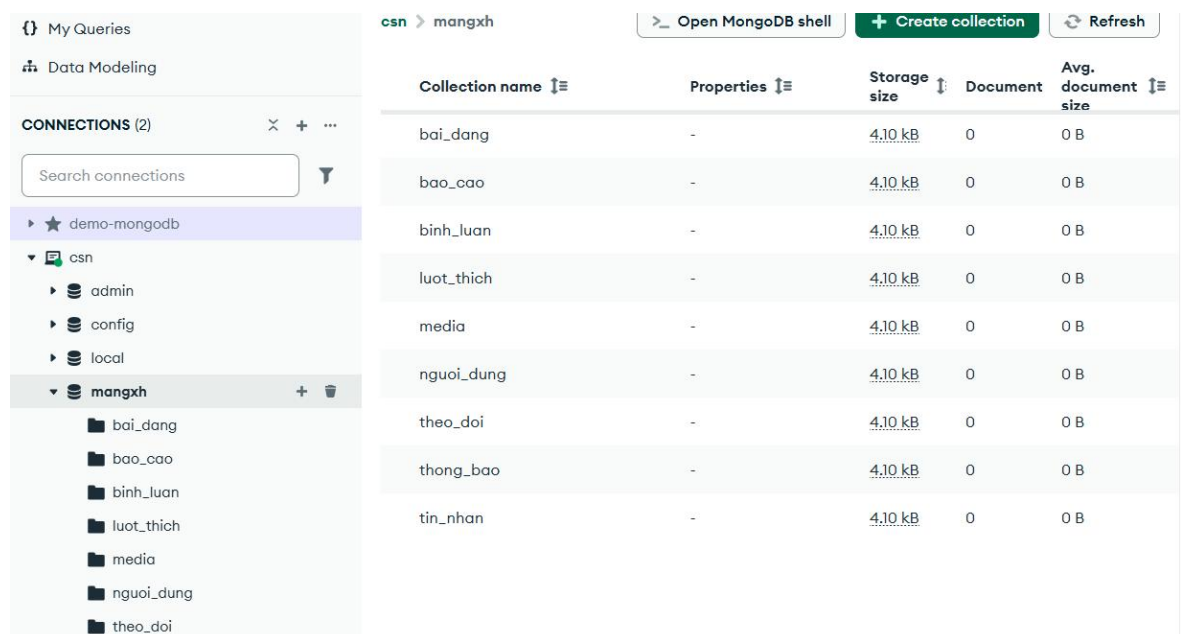
db.createCollection("thong_bao")

db.createCollection("theo_doi")

< { ok: 1 }
mangxh > |
```

Hình 3.1 Giao diện dòng lệnh khi khởi tạo Database và Collection

Sau khi thực thi script, sử dụng MongoDB Compass để kiểm tra. Kết nối tới cơ sở dữ liệu mangxh và xác nhận rằng các Collection đã được tạo.



Hình 3.2 Giao diện MongoDB Compass hiển thị Database và các Collection đã tạo thành công

CHƯƠNG 4: KẾT QUẢ NGHIÊN CỨU

4.1. Hiện thực hóa các nghiệp vụ

4.1.1. Nghiệp vụ cập nhật dữ liệu

4.1.1.1. Cập nhật thông tin người dùng

Cho phép người dùng chỉnh sửa thông tin cá nhân trên hệ thống mạng xã hội.

Kịch bản: Người dùng Trần Thị B thay đổi tên đăng nhập từ tranthib sang tranthib_new.

Câu lệnh:

```
db.nguoi_dung.updateOne
(
  { "ten_dang_nhap": "tranthib" },
  { $set: { "ten_dang_nhap": "tranthib_new" } }
)
```

Kết quả trả về từ Shell:

```
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
```

Kết quả Document (Sau khi Cập nhật):

```
{
  _id: ObjectId('700010000000000000000002'),
  ten_dang_nhap: 'tranthib_new',
  mat_khau: 'hashed_pass_2',
  email: 'tranthib@example.com',
  ho_ten: 'Trần Thị B',
  // ... (Các trường khác)
```

```
}
```

4.1.1.2. Cập nhật hàng loạt

Cho phép cập nhật đồng thời thông tin của nhiều người dùng. Đây là ưu điểm của MongoDB khi xử lý dữ liệu quy mô lớn trong hệ thống mạng xã hội.

Kịch bản: Hệ thống cập nhật tiểu sử (tieu_su) của tất cả người dùng, bằng cách nối thêm một số ngẫu nhiên.

Câu lệnh:

```
db.nguoi_dung.updateMany(  
  {},  
  [  
    {  
      $set: {  
        tieu_su: {  
          $concat: [  
            "$tieu_su",  
            " #",  
            { $toString: { $floor: { $multiply: [ { $rand: {} }, 1000 ] } } }  
          ]  
        }  
      }  
    }  
  ]  
)
```

Kết quả trả về từ Shell:

```
{  
  acknowledged: true,  
  insertedId: null,  
  matchedCount: 19,  
  modifiedCount: 19,  
  upsertedCount: 0  
}
```

Kết quả Document (Sau khi Cập nhật):

```
{
  _id: ObjectId('700010000000000000000007'),
  ten_dang_nhap: 'nguyenvang',
  mat_khau: 'hashed_pass_7',
  email: 'nguyenvang@example.com',
  ho_ten: 'Nguyễn Văn G',
  anh_dai_dien: 'https://example.com/avatar/g.jpg',
  tieu_su: 'Lập trình viên full-stack. #933',
  // ... (Các trường khác)
}
{
  _id: ObjectId('700010000000000000000008'),
  ten_dang_nhap: 'tranthih',
  mat_khau: 'hashed_pass_8',
  email: 'tranthih@example.com',
  ho_ten: 'Trần Thị H',
  anh_dai_dien: 'https://example.com/avatar/h.jpg',
  tieu_su: 'Thích du lịch và khám phá. #376',
  // ... (Các trường khác)
}
```

4.1.2. Nghiệp vụ tra cứu và tìm kiếm

4.1.2.1. Tra cứu chi tiết

Tìm chính xác hồ sơ người dùng dựa trên địa chỉ email, tiện cho việc kiểm tra thông tin, quản lý hoặc hỗ trợ người dùng.

Kịch bản: Xem thông tin chi tiết của người dùng có email tranthib@example.com.

Câu lệnh:

```
db.nguoi_dung.findOne({ "email": "tranthib@example.com" })
```

Kết quả trả về từ Shell:

```
{
  _id: ObjectId('700010000000000000000002'),
  ten_dang_nhap: 'tranthib_new',
  mat_khau: 'hashed_pass_2',
  email: 'tranthib@example.com',
  ho_ten: 'Trần Thị B',
  anh_dai_dien: 'https://example.com/avatar/b.jpg',
  // ... (Các trường khác)
}
```

4.1.2.2. Tra cứu theo tiêu chí

Tìm tất cả người dùng thỏa mãn điều kiện cụ thể.

Kịch bản: Lấy danh sách tất cả người dùng có số người theo dõi lớn hơn 500.

Câu lệnh:

```
db.nguoi_dung.find(
  { "so_nguoi_theo_doi": { $gt: 500 } },
  { ten_dang_nhap: 1, ho_ten: 1, so_nguoi_theo_doi: 1, _id: 0 }
)
```

Kết quả trả về từ Shell:

```
{
  ten_dang_nhap: 'hoangvane',
  ho_ten: 'Hoàng Văn E',
  so_nguoi_theo_doi: 560
}
{
  ten_dang_nhap: 'tranthih',
  ho_ten: 'Trần Thị H',
  so_nguoi_theo_doi: 670
}
```



```
}  
{  
  ten_dang_nhap: 'levani',  
  ho_ten: 'Lê Văn I',  
  so_nguoi_theo_doi: 800  
}
```

4.1.2.3. Tra cứu nâng cao theo Văn bản

Tìm chi tiết hồ sơ người dùng dựa trên tên đăng nhập (ten_dang_nhap).

Kịch bản: Xem hồ sơ người dùng có tên đăng nhập chứa ký tự tran.

Câu lệnh:

```
db.nguoi_dung.find({ "ten_dang_nhap": /tran/i })
```

Kết quả trả về từ Shell:

```
{  
  _id: ObjectId('700010000000000000000002'),  
  ten_dang_nhap: 'tranthib_new',  
  mat_khau: 'hashed_pass_2',  
  email: 'tranthib@example.com',  
  ho_ten: 'Trần Thị B',  
  anh_dai_dien: 'https://example.com/avatar/b.jpg',  
  // ... (Các trường khác)  
}  
{  
  _id: ObjectId('700010000000000000000008'),  
  ten_dang_nhap: 'tranthih',  
  mat_khau: 'hashed_pass_8',  
  email: 'tranthih@example.com',  
  ho_ten: 'Trần Thị H',  
  anh_dai_dien: 'https://example.com/avatar/h.jpg',  
  // ... (Các trường khác)  
}
```

4.1.3. Nghiệp vụ thống kê và báo cáo

4.1.3.1. Thống kê 1 người dùng có bao nhiêu bài đăng

Kịch bản: Thống kê số lượng và chi tiết các bài đăng của người dùng có tên đăng nhập chứa ký tự uyen.

Câu lệnh:

```
db.bai_dang.aggregate([
  // 1. Join với collection nguoi_dung
  {
    $lookup: {
      from: "nguoi_dung",
      localField: "nguoi_dung_id",
      foreignField: "_id",
      as: "nguoi_dung"
    }
  },
  // 2. Giải nén mảng nguoi_dung
  { $unwind: "$nguoi_dung" },
  // 3. Lọc chính xác tên đăng nhập
  { $match: { "nguoi_dung.ten_dang_nhap": "uyen" } },
  // 4. Gom nhóm: đếm số bài đăng + push chi tiết
  {
    $group: {
      _id: "$nguoi_dung.ten_dang_nhap",
      so_bai_dang: { $sum: 1 },
      danh_sach_bai_dang: {
        $push: {
          "Tiêu đề bài đăng": "$tieu_de",
          "Nội dung": "$noi_dung",
          "Ngày tạo": "$ngay_tao"
        }
      }
    }
  }
])
```

```
}  
}  
},  
// 5. Chọn trường hiển thị  
{  
  $project: {  
    _id: 0,  
    "Người dùng": "$_id",  
    "Số bài đăng": "$so_bai_dang",  
    "Chi tiết bài đăng": "$danh_sach_bai_dang"  
  }  
}  
])
```

Kết quả trả về từ Shell:

```
{  
  'Người dùng': 'uyen',  
  'Số bài đăng': 2,  
  'Chi tiết bài đăng': [  
    {  
      'Tiêu đề bài đăng': 'Tình cảm chân thành',  
      'Nội dung': 'love you',  
      'Ngày tạo': 2024-01-12T12:00:00.000Z  
    },  
    {  
      'Tiêu đề bài đăng': 'Cơn thèm bất chợt',  
      'Nội dung': 'Lại thèm gà nữa rồi trời ơi :((',  
      'Ngày tạo': 2024-01-14T07:20:00.000Z  
    }  
  ]  
}
```

4.1.3.2. Thống kê 1 người có bao nhiêu bình luận

Kịch bản: Thống kê số lượng và chi tiết các bình luận của người dùng có tên đăng nhập chứa ký tự uyen.

Câu lệnh:

// 3.4.3.1. Thống kê bình luận của người dùng "uyen"

```
db.binh_luan.aggregate([
  // 1. Join với collection nguoi_dung
  {
    $lookup: {
      from: "nguoi_dung",
      localField: "nguoi_dung_id",
      foreignField: "_id",
      as: "nguoi_dung"
    }
  },
  // 2. Giải nén mảng nguoi_dung
  { $unwind: "$nguoi_dung" },
  // 3. Lọc bình luận của người dùng "uyen"
  { $match: { "nguoi_dung.ten_dang_nhap": "uyen" } },
  // 4. Gom nhóm: đếm số bình luận + push chi tiết
  {
    $group: {
      _id: "$nguoi_dung.ten_dang_nhap",
      so_binh_luan: { $sum: 1 },
      danh_sach_binh_luan: {
        $push: {
          "Nội dung bình luận": "$noi_dung",
          "Ngày tạo": "$ngay_tao",
          "Bài đăng liên quan": "$bai_dang_id"
        }
      }
    }
  }
])
```

```
}  
},  
// 5. Chọn trường hiển thị  
{  
  $project: {  
    _id: 0,  
    "Người dùng": "$_id",  
    "Số bình luận": "$so_binh_luan",  
    "Chi tiết bình luận": "$danh_sach_binh_luan"  
  }  
}  
])
```

Kết quả trả về từ Shell:

```
{  
  'Người dùng': 'uyen',  
  'Số bình luận': 2,  
  'Chi tiết bình luận': [  
    {  
      'Nội dung bình luận': 'Mình cũng muốn ăn , hay là tụi mình đi chung đi!',  
      'Ngày tạo': 2024-02-14T13:55:00.000Z,  
      'Bài đăng liên quan': ObjectId('700020000000000000000001a')  
    },  
    {  
      'Nội dung bình luận': 'Ăn gà nữa đi cho đỡ thèm!',  
      'Ngày tạo': 2024-02-14T13:55:00.000Z,  
      'Bài đăng liên quan': ObjectId('700020000000000000000001a')  
    }  
  ]  
}
```

4.1.3.3. Thống kê bài đăng có bao nhiêu lượt thích và bình luận

Kịch bản: Thống kê 1 bài đăng có lượt thích và bình luận.

Câu lệnh:

```
db.bai_dang.aggregate([
  // 1. Lọc bài đăng theo tiêu đề
  {
    $match: { tieu_de: "Con thèm bắt chột" }
  },
  // 2. Join với bình luận
  {
    $lookup: {
      from: "binh_luan",
      localField: "_id",
      foreignField: "bai_dang_id",
      as: "ds_binh_luan"
    }
  },
  // 3. Hiện thị chi tiết bài đăng và bình luận
  {
    $project: {
      _id: 0,
      "Tiêu đề bài đăng": "$tieu_de",
      "Số lượt thích": "$so_luot_thich",
      "Số bình luận": { $size: "$ds_binh_luan" },
      "Chi tiết bình luận": {
        $map: {
          input: "$ds_binh_luan",
          as: "bl",
          in: {
            "Nội dung bình luận": "$$bl.noi_dung",
            "Ngày tạo": "$$bl.ngay_tao",

```

```
"Người dùng ID": "$$bl.nguoi_dung_id"
    }
  }
}
}
}
]
```

Kết quả trả về từ Shell:

```
{
  'Tiêu đề bài đăng': 'Thức uống quen thuộc',
  'Số lượt thích': 5400,
  'Số bình luận': 1,
  'Chi tiết bình luận': [
    {
      'Nội dung bình luận': 'Cà phê sữa đá best thật!',
      'Ngày tạo': 2024-02-14T12:40:00.000Z,
      'Người dùng ID': ObjectId('700010000000000000000017')
    }
  ]
}
```

4.2. Kết quả hiện thực hóa dữ liệu

Sau khi thực hiện đồ án, hệ thống cơ sở dữ liệu NoSQL quản lý người dùng mạng xã hội trên MongoDB đã được xây dựng hoàn chỉnh, chuyển đổi từ mô hình quan hệ sang mô hình Document (JSON/BSON).

Tính nhất quán: Thông tin người dùng, bài đăng, bình luận cùng các trường dữ liệu khác được nhúng trực tiếp vào Document, giúp giảm truy vấn phức tạp và tăng tốc độ truy xuất.

Cấu trúc phân cấp: Hệ thống tổ chức dữ liệu theo cấu trúc phân cấp, bao gồm thông tin cá nhân, bài đăng, bình luận, mối quan hệ giữa người dùng và các trường dữ liệu liên quan khác, giúp quản lý và truy xuất thông tin hiệu quả.

4.3. Kết quả các giao diện chức năng trên MongoDB Compass

Quản lý Document chi tiết: Xem và chỉnh sửa dữ liệu người dùng, bài đăng, bình luận và các trường liên quan khác dưới dạng cây hoặc bảng, hiển thị các Object lồng nhau để quản trị có cái nhìn tổng thể.

Thực thi bộ lọc: Hỗ trợ tìm kiếm từ đơn giản đến phức tạp, lọc người dùng và bài đăng và các trường khác theo nhiều tiêu chí.

Kiểm soát cập nhật: Các thao tác updateOne và updateMany hiển thị ngay số bản ghi bị ảnh hưởng, đảm bảo độ chính xác.

4.4. Kết quả các nghiệp vụ thống kê báo cáo

Nghiên cứu đã tận dụng Aggregation Pipeline của MongoDB để thực hiện các truy vấn phức tạp, giúp quản lý dữ liệu người dùng, bài đăng, bình luận và trường khác giữa người dùng một cách hiệu quả, đồng thời hỗ trợ các thao tác thống kê, tìm kiếm và phân tích dữ liệu trên hệ thống mạng xã hội.

4.5. Đánh giá và Phân tích kết quả

4.5.1. Về hiệu năng và tốc độ truy xuất

Tối ưu hóa đọc dữ liệu: Việc sử dụng Embedding giúp truy xuất nhanh toàn bộ thông tin người dùng, bài đăng và bình luận và các trường khác mà không cần Join.

Tối ưu hóa báo cáo: Sử dụng \$merge tạo các Collection thống kê, đảm bảo báo cáo luôn sẵn sàng và giảm tải tính toán khi truy cập dashboard.

4.5.2. Về tính linh hoạt và khả năng mở rộng

Linh hoạt lược đồ: Hệ thống cho phép thêm các trường thông tin mới (ví dụ: sở thích, trạng thái online) cho người dùng mà không ảnh hưởng đến các bản ghi cũ, tránh lãng phí không gian lưu trữ.

Khả năng mở rộng: Cấu trúc dữ liệu tương thích với cơ chế phân mảnh của MongoDB, sẵn sàng mở rộng khi số lượng người dùng và bài đăng tăng lên hàng chục ngàn bản ghi.

4.5.3. Trải nghiệm trong quá trình thực hiện

Việc sử dụng JSON giúp các lập trình viên dễ dàng làm việc vì định dạng này tương đồng với các đối tượng trong ngôn ngữ lập trình hiện đại.

Công cụ Aggregation Pipeline trực quan giúp việc xây dựng các công thức thống kê trở nên dễ kiểm soát và ít sai sót hơn so với viết các câu lệnh SQL dài.

4.6. Ưu điểm và nhược điểm

4.6.1. Ưu điểm

Hiệu năng truy xuất cao: Dữ liệu người dùng, bài đăng, bình luận và các trường khác được lưu trực tiếp trong Document, giảm thiểu các phép Join và tăng tốc truy vấn.

Tính linh hoạt cao: Có thể thêm trường mới cho từng người dùng mà không ảnh hưởng đến dữ liệu hiện có, phù hợp với môi trường mạng xã hội thay đổi liên tục.

Khả năng xử lý mạnh mẽ: Aggregation Pipeline cho phép tổng hợp, phân tích bài đăng, bình luận và tương tác một cách tự động và chính xác.

Cấu trúc dữ liệu trực quan: Dữ liệu JSON/BSON trực quan, dễ đọc và tích hợp với ứng dụng, giảm công sức xử lý cho lập trình viên.

4.6.2. Nhược điểm

Dư thừa dữ liệu: Một số thông tin người dùng hoặc bài đăng được lưu trực tiếp trong nhiều Document, làm tăng dung lượng lưu trữ.

Phức tạp trong cập nhật dữ liệu: Khi thay đổi thông tin chung (ví dụ: tên người dùng, trạng thái), cần cập nhật trên tất cả Document liên quan, dễ gây không đồng bộ nếu không quản lý cẩn thận.

Thiếu các ràng buộc toàn vẹn mặc định: MongoDB không có khóa ngoại, việc đảm bảo dữ liệu giữa người dùng, bài đăng và trường khác, MongoDB đẩy trách nhiệm kiểm soát tính toàn vẹn dữ liệu về phía ứng dụng. Nếu không kiểm soát kỹ ở tầng Backend, có thể dẫn đến việc tham chiếu đến dữ liệu không tồn tại.

Chưa tối ưu dung lượng lưu trữ: Tên các trường lặp lại trong mỗi Document, sử dụng bộ nhớ nhiều hơn so với mô hình bảng quan hệ.

CHƯƠNG 5: KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

5.1. Kết luận

Về mặt dữ liệu: Hệ thống đã xây dựng thành công cấu trúc hồ sơ người dùng trên MongoDB.

Về mặt nghiệp vụ: Hiện thực hóa đầy đủ các thao tác quản lý dữ liệu, bao gồm cập nhật thông tin người dùng hàng loạt và tra cứu dữ liệu linh hoạt dựa trên nội dung văn bản.

Về mặt thống kê: Sử dụng Aggregation Pipeline để xây dựng các báo cáo tự động, cung cấp các chỉ số về số lượng người dùng, bài đăng và cúac trường khác với độ chính xác cao.

Về mặt hiệu năng: Kết quả cho thấy ưu thế vượt trội của MongoDB về tốc độ truy xuất dữ liệu và khả năng mở rộng cấu trúc so với các hệ quản trị cơ sở dữ liệu quan hệ truyền thống.

5.2. Hướng phát triển

Xây dựng hệ thống Backend hoàn chỉnh: Xây dựng máy chủ bằng Node.js kết nối trực tiếp với MongoDB, xử lý logic và cung cấp dữ liệu cho ứng dụng Web và Mobile.

Triển khai kiến trúc RESTful API hoặc GraphQL: Sử dụng RESTful API hoặc GraphQL để truy xuất dữ liệu an toàn, dễ mở rộng và tích hợp các dịch vụ mới mà không thay đổi cấu trúc cốt lõi.

Tích hợp cơ chế xác thực và bảo mật: Áp dụng JWT hoặc OAuth2 để quản lý phiên đăng nhập và phân quyền người dùng, đảm bảo mỗi cá nhân chỉ thao tác trong phạm vi được phép

Tự động hóa báo cáo và phân tích: Tận dụng Aggregation Pipeline để tạo các thống kê thời gian thực về người dùng, bài đăng, bình luận và tương tác, trả dữ liệu dưới dạng JSON cho giao diện trực quan.

Triển khai trên nền tảng đám mây: Đưa hệ thống lên các nền tảng như AWS, Google Cloud hoặc Azure, kết hợp Sharding để mở rộng quy mô, đảm bảo vận hành ổn định với lượng người dùng lớn.

DANH MỤC TÀI LIỆU THAM KHẢO

- [1] Meier and M. Kaufmann, SQL & NoSQL Databases: Models, Languages, Consistency Options and Architectures for Big Data Management. Springer Vieweg, 2019.
- [2] S. Bradshaw, E. Brazil, and K. Chodorow, MongoDB: The Definitive Guide: Powerful and Scalable Data Storage. O'Reilly Media, 2019.
- [3] Celko, Joe. Joe Celko's Complete Guide to NoSQL: What Every SQL Professional Needs to Know about Nonrelational Databases. Morgan Kaufmann, 2014.
- [4] Phan thị Phương Nam, Bài tập thực hành SQL và NoSQL, Trường Đại học Trà Vinh, 2024.
- [5] K. Banker et al., MongoDB in Action, 2nd ed. Manning Publications, 2016.
- [6] R. Copeland, MongoDB Applied Design Patterns. Sebastopol, CA: O'Reilly Media, Inc., 2013.
- [7] https://www.w3resource.com/w3r_images/mongodb-document-ollection.png